```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_captioning_path = kagglehub.dataset_download('rehmang110/captioning')

print('Data source import complete.')
```

```
import numpy as np
import torch
import os
from pathlib import Path
from PIL import Image
import shutil
import cv2
import matplotlib.pyplot as plt
```

```
print(torch.__version__)
```

```
2.6.0+cu124
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype= torch.float16

else:
    device = "cpu"
    dtype = torch.float32

print("the device is :",device)
print("the data type is :", dtype)
```

```
the device is : cpu
the data type is : torch.float32
```

```
# move the to working directory
src_path = Path("/kaggle/input/captioning")
dest_path = Path("/kaggle/working/test_images")
dest_path.mkdir(parents=True , exist_ok =True)

for file in src_path.iterdir():
    shutil.copy(file , dest_path / file.name)
```

```
#  torch has the hub from where we load the MIDAS model for our use
midas = torch.hub.load("intel-isl/MiDaS", "DPT_Hybrid")
```

```
Using cache found in /root/.cache/torch/hub/intel-isl_MiDaS_master
```

```
# before feeding the image to the model we have to transform it and for that we have the midas transform
midas_transforms = torch.hub.load("intel-isl/MiDaS", "transforms")
transform = midas_transforms.dpt_transform
```

```
Using cache found in /root/.cache/torch/hub/intel-isl_MiDaS_master
```

```
image_path = "/kaggle/working/test_images/pexels-soc-nang-d-ng-2150345854-33267352.jpg"
image = cv2.imread(image_path)                                # Reads in BGR
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)          # Convert to RGB (NumPy array)

# show the np array image
```

```
plt.imshow(image)
plt.axis("off")
plt.title("RGB Image")
plt.show()
```


```
# accecpt the image as an input np_array
input_tensor = transform(image)
```

```
with torch.no_grad():
    prediction = midas(input_tensor)
```

```
print("Prediction shape:", prediction.shape)
print("Min:", prediction.min().item())
print("Max:", prediction.max().item())
print("Mean:", prediction.mean().item())
```

```
Prediction shape: torch.Size([1, 384, 576])
Min: 0.0
Max: 2293.5380859375
Mean: 717.4137573242188
```

```
depth_map = prediction.squeeze().cpu().numpy()
print("Depth map shape:", depth_map.shape)
print("Depth map dtype:", depth_map.dtype)
```

```
Depth map shape: (384, 576)
Depth map dtype: float32
```

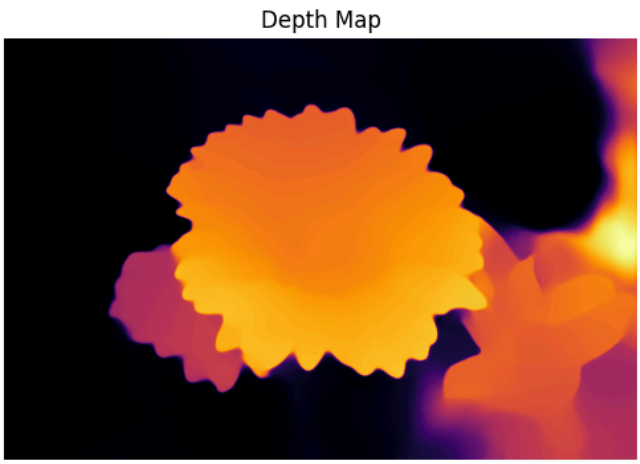
```
# Normalize for display (0–1)
depth_normalized = (depth_map - depth_map.min()) / (depth_map.max() - depth_map.min())

# Show original and depth side by side
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(depth_normalized, cmap='inferno')
plt.title("Depth Map")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Normalize depth map to 0-255 for colormap
depth_norm = cv2.normalize(depth_map, None, 0, 255, cv2.NORM_MINMAX)
depth_uint8 = depth_norm.astype(np.uint8)

# Apply colormap (e.g. inferno, magma, jet, etc.)
depth_colored = cv2.applyColorMap(depth_uint8, cv2.COLORMAP_TURBO)  # shape: [H, W, 3], BGR

# Convert depth_colored from BGR → RGB to match original image
depth_colored = cv2.cvtColor(depth_colored, cv2.COLOR_BGR2RGB)

# Resize original image if needed (just in case)
image_resized = cv2.resize(image, (depth_colored.shape[1], depth_colored.shape[0]))

# Blend the heatmap and original image
overlay = cv2.addWeighted(image_resized, 0.6, depth_colored, 0.4, 0)

# Show result
plt.figure(figsize=(10, 5))
plt.imshow(overlay)
plt.title("Depth Overlay on RGB")
plt.axis('off')
plt.show()
```